

Cybersecurity Architecture & Defensive Controls

Vibarua Marketplace Platform - Technical Defense & Security Engineering Documentation

Focus Domain: Application Security (AppSec) & Secure SDLC

Framework Alignment: OWASP Top 10 (2021) & NIST CSF

Technology Stack: Node.js, Express, MongoDB, Cloudinary, React 19

Document Scope: End-to-End Defensive Controls Specification

1 Identity & Access Management (IAM)

User Authentication & Stateful Access Verification

AUTHENTICATION

Guarantees that actors authenticate with valid credentials before accessing protected operations.

- **Credential Verification:** Handled via `/api/auth/login` with email and password checks.
- **Pre-Authentication Account Standing Check:** Prior to generating a token, the backend inspects account standing. Suspended accounts (`isBlocked: true`) or deactivated accounts (`isActive: false`) are rejected with an explicit `403 Forbidden` and audit logged, preventing suspended users from obtaining fresh access tokens.

Cryptographic Password Security & Hashing

CRYPTOGRAPHIC STORAGE

Eliminates plaintext credential storage and defends against offline brute-force attacks.

- **Algorithm:** Implemented via `bcryptjs` using a salt cost factor of **10 rounds** with cryptographically secure random salt generation.
- **Threats Mitigated:** Rainbow table lookups, dictionary attacks, and credential exposure in the event of database exfiltration.
- **Query Projection:** Mongoose queries systematically exclude password hashes using `.select('-password')` across all user and admin endpoints.

Role-Based Access Control (RBAC) & Principle of Least Privilege

AUTHORIZATION

Enforces least privilege by separating system roles into three distinct authorization tiers.

- **Defined Roles:** `client`, `worker`, and `admin`.
- **Route Middleware Enforcement:** Enforced via `authorize('admin')` middleware. Non-admin roles attempting to access administrative controllers receive an immediate `403 Forbidden`.

JWT Authorization & Live Database Revocation Check

SESSION SECURITY

Provides stateless token authorization augmented with real-time database validation.

- **Signature:** Tokens are digitally signed using HMAC-SHA256 (HS256) with a 30-day expiration window.
- **Live Invalidation Check:** Standard stateless JWTs suffer from "token linger" (tokens remain valid even after an admin bans a user until token expiry). In this system, the protect middleware checks the live MongoDB record on **every request**:
 - Validates user existence in the database.
 - Ensures `user.isBlocked === false` and `user.isActive !== false`.
 - **Outcome:** When an admin blocks an account, active sessions and tokens are **immediately neutralized**.

2 Input Validation, Data Sanitization & Type Enforcement

Server-Side Schema Validation & Type Constraints (Mongoose)

INJECTION PREVENTION

Authoritative server-side validation rejecting invalid, malformed, or malicious data payloads before reaching database storage:

- **Normalization (lowercase: true):** Normalizes email addresses to lowercase, preventing case-sensitivity authentication bypasses and duplicate account spoofing (e.g. `User@domain.com` vs `user@domain.com`).
- **Whitespace Sanitization (trim: true):** Trims leading/trailing whitespace across all text fields (full name, email, bio, profession, comments), neutralizing whitespace manipulation attacks.
- **Length Boundaries:** Enforces minimum password length (`minlength: 6`) and maximum string length limits (`maxlength: 500` on bio and review comments), preventing buffer stuffing and database payload exhaustion.
- **Strict Value Enumeration (Whitelisting):** Restricts user roles to `['client', 'worker', 'admin']` and review ratings to numeric values between 1 and 5. Any unexpected value triggers a Mongoose validation rejection.
- **Numeric Range Guards:** Restricts experience to `min: 0` and rating calculations to `min: 0, max: 5`, eliminating negative integer manipulation.

Client-Side Form Validation & Real-Time Strength Metering

CLIENT-SIDE DEFENSE

- **Interactive Password Strength Meter:** Evaluates passwords against 6 criteria (`length ≥ 8`, `length ≥ 12`, uppercase letters, lowercase letters, numbers, and special symbols) with real-time visual color feedback.
- **Confirmation Matching:** Validates password confirmation equality prior to network dispatch (`password === confirmPassword`).
- **Input Bounds:** HTML5 input types (`type="email"`, `type="number"`) and native field constraints.

Query Parameter Sanitization & Boundary Handling

PARAMETER DEFENSE

- **Safe Integer Parsing:** Query parameters for pagination are defensively parsed with strict fallback limits (`parseInt(req.query.page) || 1, parseInt(req.query.limit) || 10`) preventing NaN or infinite pagination queries.
- **Search Sanitization:** Administrative user and audit searches sanitize user input strings before regex evaluation.

3 Threat Mitigation & Defensive Architecture

Multi-Tiered Rate Limiting & Brute-Force Defense

DOS / ANTI-BRUTE-FORCE

Configured using `express-rate-limit` with tiered thresholds mapped to endpoint sensitivity:

- **Authentication Limiter (`authLimiter`):** Imposes a strict limit of **5 requests per 15 minutes** per IP address on authentication endpoints (`/api/auth/login` and `/api/auth/register`). Prevents automated credential stuffing, password spraying, and dictionary attacks.
- **General API Limiter (`generalLimiter`):** Restricts traffic across all API endpoints to **100 requests per 15 minutes** per IP, protecting the Node.js event loop against Layer-7 resource starvation and Denial of Service (DoS).
- **Automated Threat Event Trigger:** Exceeding the authentication threshold automatically generates a security audit log (`RATE_LIMIT_EXCEEDED`) capturing the offending IP address, requested path, method, and User-Agent.

4 Secure File Uploads & Media Handling

Cloud-Isolated File Uploads & MIME Whitelisting

FILE UPLOAD SECURITY

Prevents file upload vulnerabilities, Remote Code Execution (RCE), and server filesystem compromise:

- **Strict Extension / Format Whitelist:** Only image formats are accepted (`['jpg', 'jpeg', 'png', 'gif', 'webp']`). Executable extensions (`.exe, .php, .sh, .js, .py, .html`) are strictly rejected.
- **Decoupled Cloud Storage:** Uploaded media is streamed directly to Cloudinary CDN storage rather than stored on the local web server filesystem, completely eliminating **Local File Inclusion (LFI)** and web shell attacks.
- **Image Re-encoding & Neutralization:** Images undergo server-side transformation (`{ width: 500, height: 500, crop: 'fill', quality: 'auto' }`). Re-encoding strips embedded EXIF metadata payloads, malicious polyglot scripts, and ImageTragick exploits.
- **Multer Error Boundary:** Dedicated error handling middleware intercepts file size and upload exceptions gracefully without crashing the server process.

5 Account Integrity, Suspension & Soft Deactivation

Administrative Account Suspension (Block / Unblock)

INCIDENT CONTAINMENT

Enables administrators to immediately isolate compromised or abusive user accounts.

- **Granular Tracking:** Persists `isBlocked` (Boolean), `blockReason` (administrative reason), `blockedAt` (timestamp), and `blockedBy` (administrator ID).
- **Self-Lockout Prevention:** Hardcoded logic prevents administrators from suspending or deactivating their own account (`req.userId === targetId`), preventing administrative lockout.

Soft Deactivation / Soft Deletion (Forensics & Non-Repudiation)

DATA PRESERVATION

Adopts an account deactivation pattern (`isActive: false`) rather than permanent database deletion (`findByIdAndDelete`).

- **Preservation of Forensic Evidence:** Hard deletion destroys historical relational integrity. Soft deactivation preserves job records, review logs, and audit trails for post-incident investigation.
- **Non-Repudiation:** Malicious actors cannot erase evidence of illicit transactions or policy violations by attempting account deletion.

6 Security Auditing, Observability & Sensitive Data Redaction

Centralized Security Audit Logging Subsystem (AuditLog)

ACCOUNTABILITY & SIEM

An immutable audit logging subsystem capturing security-significant lifecycle events into MongoDB.

- **Tracked Actions:** LOGIN_SUCCESS, LOGIN_FAILED, USER_REGISTERED, LOGOUT, ACCOUNT_BLOCKED, ACCOUNT_UNBLOCKED, ACCOUNT_DEACTIVATED, ACCOUNT_REACTIVATED, RATE_LIMIT_EXCEEDED, ADMIN_ACTION.
- **Captured Forensic Telemetry:** Actor User ID, real client IP address (supporting proxy/load-balancer headers via `x-forwarded-for`), User-Agent header, UTC timestamp, outcome status (success, failure, warning), and context metadata.
- **Compound Database Indexing:** Indexed on (`userId`, `timestamp`) and (`action`, `timestamp`) for high-performance forensic querying.
- **Append-Only Integrity:** Audit logs can only be created by system events; there are no public or administrative update/delete endpoints.

Automated Recursive Credential Redaction (`sanitizeDetails`)

PRIVACY & DEFENSE IN DEPTH

Mitigates log poisoning and accidental credential exposure in persistent storage or log drains:

- **Recursive Inspection:** Recursively traverses nested objects and arrays.
- **Key Sanitization:** Masks keys matching `/password|token|jwt|secret|apiKey|auth|bearer|cookie|credential/i` to `[REDACTED]`.
- **Signature Detection:** Identifies raw Bearer authorization strings and JWT signatures (`eyJ...`) and replaces them with `[REDACTED_TOKEN]`.

7 Network, Transport & Configuration Hardening

CORS Whitelisting, Error Masking & Environment Isolation

INFRASTRUCTURE HARDENING

- **Strict CORS Policy:** Restricted to explicit trusted client origins (`http://localhost:5173`, production Vercel frontend) with `credentials: true`.
- **Information Leakage Prevention:** Generic error responses ("Invalid credentials", "Server error") prevent user enumeration and conceal internal stack traces.
- **Secret Isolation:** All credentials (MongoDB URI, JWT secret, Cloudinary keys) are isolated in `.env` and excluded from version control via `.gitignore`.
- **DNS Security:** Explicit DNS resolution (`8.8.8.8`, `8.8.4.4`) with IPv4 priority prevents DNS rebinding vulnerabilities during Atlas SRV queries.

8 Cybersecurity Master Matrix (Presentation Reference)

SECURITY CONTROL	IMPLEMENTATION MECHANISM IN PROJECT	THREAT / VULNERABILITY MITIGATED
Input Validation & Sanitization	Mongoose schema constraints (lowercase, trim, minlength, maxlength, strict enums, numeric ranges)	Injection attacks, unexpected data tampering, buffer stuffing, casing bypasses
Password Hashing	bcryptjs (Cost factor 10 rounds) + schema .select('-password')	Plaintext credential exposure, rainbow table lookups, database breach cracking
Authentication & Session Tokens	HMAC-SHA256 JWT + Live database standing verification in protect	Unauthorized access, session hijacking, stale token lingering post-ban
Role-Based Access Control (RBAC)	Role segregation (client, worker, admin) via route middleware	Privilege escalation, Broken Object Level Authorization (BOLA)
Brute-Force & DoS Mitigation	authLimiter (5 req / 15 min) + generalLimiter (100 req / 15 min)	Credential stuffing, password spraying, automated dictionary attacks, Layer-7 DoS
Secure File Uploads	Extension whitelist (jpg, png, webp) + Cloudinary CDN + image re-encoding	Remote Code Execution (RCE), Local File Inclusion (LFI), web shells, EXIF malware
Account Suspension	Administrative block toggle + immediate token invalidation	Compromised/abusive account containment, terms enforcement
Soft Deactivation	Soft deletion pattern (isActive: false) preserving records	Evidence tampering, anti-forensics, loss of relational audit integrity
Security Audit Logging	Indexed, append-only AuditLog collection with IP & User-Agent	Lack of accountability, non-repudiation vulnerabilities
Sensitive Data Redaction	Recursive regex pattern matching in sanitizeDetails()	Log poisoning, sensitive credential leakage in server logs or monitoring
Self-Lockout Protection	Hardcoded check prohibiting admins from suspending/deactivating self	Administrative privilege suicide, management plane DoS
CORS & Transport Hardening	Whitelisted trusted origins + .env secret isolation + error masking	Cross-Origin data theft, credential leakage, user enumeration